

8.1 Loopy Recurrent Neural Networks

A hidden state that reads the text one token at a time.

These notes walk through the lecture slides. The original deck is unchanged; use **(slides)** on the course hub if you want that PDF.

1. Networks without memory

A dense net and a CNN see an input, emit an output, and forget. Nothing is stored between examples, and (in a plain CNN) nothing is stored between distant tokens except what a fixed window can cover.

Reading is not like that. You hold what came earlier and update it. **Recurrent neural networks (RNNs)** do the same: they keep a **state** and revise it at every token.

Chollet's picture: biological systems process a stream and maintain an internal model. An RNN is the engineering version of that idea, not a model of a brain.

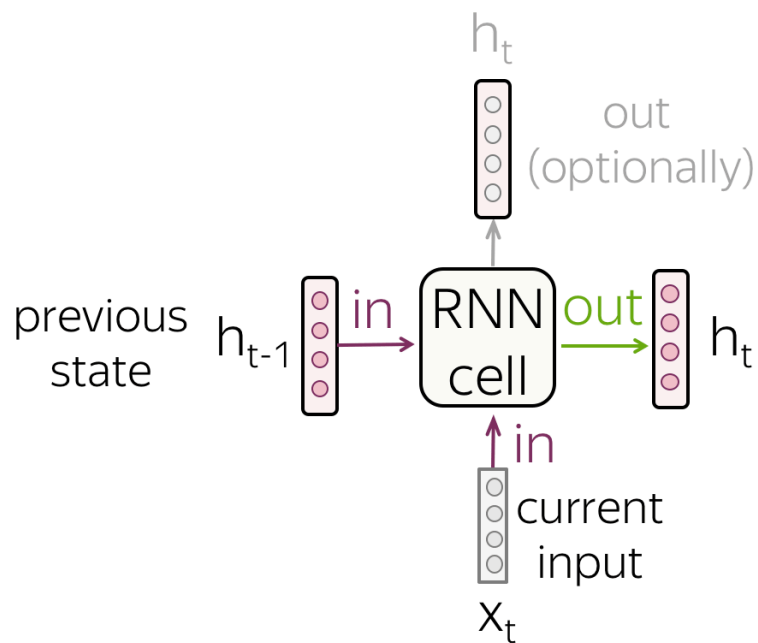
Sources: *Practical Natural Language Processing*, *Natural Language Processing in Action*, and Voita's notes. StatQuest's RNN video is a useful watch.

2. The cell

At step (t) the cell receives

- a new input ($\mathbf{x}_t$) (usually a token embedding), and
- the previous state ($\mathbf{h}_{t-1}$).

It emits a new state ($\mathbf{h}_t$) that is supposed to hold **both**. That state is the input to the next step.



The state is **reset** between independent sequences (two reviews in a batch). Each review is still one data point. What changed is that the net **loops** inside that data point.

Unrolled in time, the same weights appear at every step. That is weight sharing along the sequence, analogous to a CNN's shared filter, but the path is a chain, not a slide.

3. Vanilla RNN

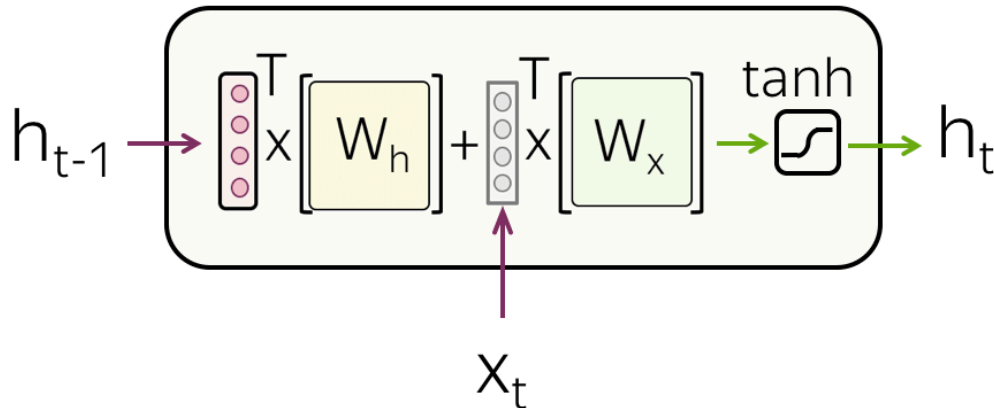
The simplest cell is a linear mix plus ($\tanh$):

$$[\mathbf{h}]_t = \tanh(\mathbf{h}_{t-1} W_h + \mathbf{x}_t W_x)$$

(Lecture writes (W_t) for the input map. Same idea.)

Vanilla RNN

$$h_t = \tanh(h_{t-1}W_h + x_tW_x)$$



Vanishing and exploding gradients. Backprop through twenty ($\tanh$) steps multiplies twenty Jacobians. The signal to early tokens dies or blows up. Vanilla cells struggle on long reviews.

LSTM and **GRU** add **gates**: learned valves that decide what to keep, write, and forget. They do not remove the problem; they make long-range training feasible. This course will treat them as drop-in cells with the same loop around them.

4. Runtime, GPUs, and unrolling

A tiny RNN is a chain of small matrix multiplies inside a `for` loop. That chain does **not** parallelize well. On a multicore **CPU** it can beat a GPU. A large recurrent layer can use a GPU.

In Keras, default `LSTM` / `GRU` on GPU call **cuDNN**. Those kernels are fast and rigid. **Recurrent dropout** is not in the default kernel; Keras falls back to a slower TensorFlow implementation with the same math.

If you cannot use cuDNN, **unrolling** the loop (write the (T) steps into the graph) can help the compiler. It also makes the graph huge. Use it when sequences are short and you have measured a benefit.

None of this changes the math. It changes whether the job finishes before the session ends.

5. What this note is not

This is not yet a classifier. You have a sequence of states ($\mathbf{h}_1, \dots, \mathbf{h}_T$). Note **8.2** says which state(s) you turn into a document label: last state, stacked layers, or a bidirectional pair.

Do not use a vanilla RNN as your first production model. Use LSTM/GRU, or skip to a transformer later in the course. Do use this note to explain **why** a loop exists.

Unroll once on paper. Three tokens (x_1, x_2, x_3), start from ($\mathbf{h}_0 = \mathbf{0}$). Compute ($\mathbf{h}_1$), then ($\mathbf{h}_2$), then ($\mathbf{h}_3$) with the **same** (W_h, W_x). The classifier in note **8.2** will sit on ($\mathbf{h}_3$) (or on a bidirectional pair). If you cannot do that by hand for ($d=2$), the Keras layer will stay a black box.

6. CNN versus RNN, one line

A CNN looks at a **fixed** neighborhood at every position, in parallel. An RNN looks at **everything so far**, in order, and cannot be fully parallelized across time. That is the accuracy / speed tradeoff you are choosing.

7. Practice

1. A CNN kernel of size 5 and a vanilla RNN both read a 40-token sentence. Which one can, in principle, mix token 1 with token 40 in a single representation, and how?
2. Why do we reset ($\mathbf{h}$) between two training documents?
3. You add recurrent dropout to an LSTM on a GPU and training becomes much slower. What happened?